
Bot Discord Państwa – Miasta

Dokumentacja API

Autor: Kamil Bujak

E-mail: kamil.bujak04@gmail.com

Rok: 2025/2026

Spis treści

1	Przegląd	2
1.1	Stack technologiczny	2
2	Discord API (discord.py)	3
2.1	Konfiguracja bota	3
2.2	Komendy — interfejs użytkownika	3
2.3	Embedy Discord	4
2.4	Interaktywne przyciski (discord.ui.View)	4
2.5	Wiadomości prywatne (DM)	4
3	API walidacji	6
3.1	Architektura hybrydowa	6
3.2	Schemat walidacji	6
3.3	Normalizacja tekstu	7
3.4	Zewnętrzne API — szczegóły	7
3.4.1	REST Countries — Państwa	7
3.4.2	Nominatim (OpenStreetMap) — Miasta	8
3.4.3	Polska Wikipedia — Zwierzęta i Rośliny	8
4	API bazy danych (SQLAlchemy 2.0)	9
4.1	DatabaseHandler — metody publiczne	9
4.2	Reset sezonowy	9
5	Voice API (TTS)	10
5.1	System ogłoszeń głosowych	10
5.2	Ogłoszenia w grze	11
5.3	Kolejność zdarzeń w rundzie	11
6	Przepływ danych — diagram sekwencji	12
A	Zależności projektu	14

1. Przegląd

1.1 Stack technologiczny

Komponent	Technologia	Uwagi
Platforma	Discord API	Komendy, embedy, role, DM, voice, UI buttons
Język	Python 3.11+	Async/await, type hints
ORM	SQLAlchemy 2.0	Mapped[] — poprawne typy statyczne
Baza danych	SQLite	Lokalny plik <code>bot.db</code>
TTS	gTTS + pyttsx3	Polski głos; pyttsx3 jako fallback offline
Walidacja	Listy + API	Lokalne listy primary; REST Countries, Nominatim, Wikipedia fallback
UI	discord.py Views	Przyciski zaskarżania, głosowania, następnej rundy

Tabela 1.1: Stack technologiczny

Bot wymaga tylko tokenu Discord (`BOT_TOKEN` w pliku `.env`). Walidacja korzysta najpierw z lokalnych zbiorów słów (offline, $O(1)$), a zewnętrzne API (REST Countries, Nominatim, Wikipedia) są odpytywane tylko gdy słowa nie ma na lokalnej liście.

2. Discord API (discord.py)

2.1 Konfiguracja bota

```
intents = discord.Intents.default()
intents.message_content = True      # odczyt tresci wiadomosci
intents.members = True              # dostep do listy czlonkow

bot = commands.Bot(command_prefix="/", intents=intents)
```

Listing 2.1: Inicjalizacja bota

2.2 Komendy — interfejs użytkownika

Komenda	Parametry	Wymagania	Odpowiedź
/start	—	Serwer, kanał text	Embed lobby z kategoriami i punktacją
/dołącz	—	Aktywne lobby	Embed: potwierdzenie dołączenia
/stop	—	Aktywna gra	Embed: zatrzymanie gry
/statystyki	[@gracz]	—	Embed: statystyki gracza
/ranking	—	Serwer	Embed: top 3 sezonu
/sezon	—	—	Embed: data resetu sezonu
/pomoc	—	—	Lista komend
/reset	—	Administrator	Reset sezonu
/ustawienia	[klucz wart. ...]	Administrator	Embed: aktualne ustawienia

Tabela 2.1: Komendy bota jako endpointy

Komenda /ustawienia przyjmuje wiele parametrów naraz:

```

/ustawienia                                # pokaz aktualne wartosci
/ustawienia rundy 5 czas 60 lobby 30      # zmien kilka naraz
/ustawienia czas 45                        # zmien jeden parametr

```

Listing 2.2: Przykłady ustawień

2.3 Embedy Discord

Kolor	RGB	Użycie
Niebieski	88, 101, 242	Informacje, rundy, lobby, następna runda
Zielony	Discord green	Potwierdzenie startu, zaakceptowane zaskarżenie
Czerwony	Discord red	Błędy, anulowanie, odrzucone zaskarżenie
Złoty	Discord gold	Wyniki końcowe, zwycięzca
Żółty	Discord yellow	Faza zaskarżania, głosowanie

Tabela 2.2: Schemat kolorów embedów

2.4 Interaktywne przyciski (discord.ui.View)

Po każdej rundzie bot wyświetla widoki z przyciskami:

Klasa	Przyciski	Opis
AppealView	Zaskarż odpowiedź	Tylko właściciel może kliknąć swój przycisk (60s)
VoteView	Akceptuj / Odrzuć	Pozostali gracze głosują; właściciel nie głosuje (30s)
NextRoundView	Następna runda	Tylko organizator; ostatnia runda: Zakończ rozgrywkę

Tabela 2.3: Interaktywne widoki Discord UI

2.5 Wiadomości prywatne (DM)

Bot wysyła DM do każdego gracza na początku rundy oraz przypomnienia proporcjonalne do czasu rundy:

```
# warn1: 50% pozostalego czasu
warn1_remaining = ROUND_TIME // 2

# warn2: ok. 15% pozostalego czasu (min. 10s)
warn2_remaining = max(10, ROUND_TIME // 6)

# Przykład dla ROUND_TIME = 60s:
# po 30s -> "Zostalo 30 sekund!"
# po 50s -> "Zostalo 10 sekund!"

# Przykład dla ROUND_TIME = 300s (5 min):
# po 150s -> "Zostalo 2 minuty i 30 sekund!"
# po 250s -> "Zostalo 50 sekund!"
```

Listing 2.3: Proporcjonalne przypomnienia

Każde przypomnienie wysyła jednocześnie:

- DM do graczy, którzy jeszcze nie wysłali odpowiedzi
- Ogłoszenie głosowe przez voice TTS

3. API walidacji

3.1 Architektura hybrydowa

Walidacja działa dwuetapowo:

1. **Lokalna lista** (primary) — zbiór `set`, sprawdzanie $O(1)$, bez sieci. Jeśli odpowiedź jest na liście, jest akceptowana natychmiast.
2. **Zewnętrzne API** (fallback) — wywoływane tylko gdy słowa nie ma na liście. Timeout 5 sekund; błąd lub brak odpowiedzi → **False** (nigdy fail-open).

Kat.	Lokalna lista	API fallback	Uwagi
Państwo	<code>countries_pl.py</code> ~250	REST Countries v3.1	Dokładne dopasowanie pol. nazwy
Miasto	<code>cities_pl.py</code> ~1155	Nominatim (OSM)	Max 1 req/s; dokładna nazwa
Imię	<code>names_pl.py</code> + zdrobnienia	brak	Tylko lista
Zwierzę	<code>animals_pl.py</code>	Polska Wikipedia	Kategorie: Ssaki, Ptaki...
Roślina	<code>plants_pl.py</code>	Polska Wikipedia	Kategorie: Rośliny, Drzewa...

Tabela 3.1: Źródła walidacji według kategorii

3.2 Schemat walidacji

```
async def validate_answer(category, answer, letter):  
    # 1. Podstawowe sprawdzenia (bez IO)  
    fail = _basic_check(answer, letter) # dlugosc, litera,  
        znaki  
    if fail:  
        return fail
```

```
# 2. Cache      wynik zwrócony natychmiast
cache_key = (category, _norm(answer), _norm(letter))
if cache_key in _cache:
    return _cache[cache_key]

# 3. Lokalna lista O(1)
if _lookup(answer, LOCAL_SETS[category]):
    return True, f"OK {answer}"

# 4. Zewnętrzne API (tylko gdy nie ma na liście)
result = await API_HANDLERS[category](answer, letter)
_cache[cache_key] = result
return result
```

Listing 3.1: Przepływ walidacji

3.3 Normalizacja tekstu

```
def _norm(text: str) -> str:
    # UWAGA: 'ł' nie dekomponuje się przez NFKD!
    text = text.lower().replace("ł", "l")
    return (
        unicodedata.normalize("NFKD", text)
        .encode("ascii", "ignore")
        .decode("ascii")
    )
# Efekty:
# "Krak w" -> "krakow"
# "Pawe " -> "pawel" (nie "pawe" jak bez replace!)
# " d " -> "lodz"
```

Listing 3.2: Funkcja normalizacji — obsługa polskich znaków

3.4 Zewnętrzne API — szczegóły

3.4.1 REST Countries — Państwa

Endpoint: `/v3.1/translation/{name}` oraz `/v3.1/name/{name}?fullText=true`

Odpowiedź jest akceptowana tylko gdy polska nazwa zwróconego kraju (`translations.pol.common`) **dokładnie** odpowiada wpisanej nazwie po normalizacji.

3.4.2 Nominatim (OpenStreetMap) — Miasta

Endpoint: `/search?q={name}&namedetails=1&accept-language=pl`

Bot sprawdza pola `name`, `name:pl`, `alt_name` i `display_name`. Wymagane dokładne dopasowanie do co najmniej jednego z nich. Stosowany semaforem (`asyncio.Semaphore(1)`) do przestrzegania limitu 1 req/s.

3.4.3 Polska Wikipedia — Zwierzęta i Rośliny

Endpoint: `/w/api.php?action=query&titles={name}&prop=categories`

Kategoria	Słowa kluczowe w kategoriach Wikipedii
Zwierzę	ssaki, ptaki, gady, płazy, ryby, owady, pajęczaki, kręgowce, bezkręgowce
Roślina	rośliny, drzewa, krzewy, kwiaty, zioła, trawy, grzyby, mchy, paprocie

Tabela 3.2: Słowa kluczowe kategorii Wikipedii

Obsługiwane są przekierowania (np. *Panda* → *Panda wielka* → kategoria Ssaki — akceptowane). Jeśli nastąpiło przekierowanie z wpisanej nazwy, sprawdzanie tytułu artykułu jest pomijane — samo przekierowanie potwierdza poprawność nazwy.

4. API bazy danych (SQLAlchemy 2.0)

4.1 DatabaseHandler — metody publiczne

Metoda	Zwraca	Opis
<code>save_player_stats()</code>	<code>None</code>	Zapisuje/aktualizuje statystyki gracza
<code>get_player_stats()</code>	<code>PlayerModel None</code>	Pobiera statystyki gracza
<code>get_leaderboard()</code>	<code>list[PlayerModel]</code>	Top N graczy serwera
<code>save_game()</code>	<code>None</code>	Zapisuje log zakończonej gry
<code>get_last_reset()</code>	<code>datetime None</code>	Czas ostatniego resetu sezonu
<code>reset_season()</code>	<code>None</code>	Zeruje wszystkie statystyki
<code>days_until_next_reset()</code>	<code>int</code>	Dni do następnego autoreset
<code>needs_reset()</code>	<code>bool</code>	Czy minęło 10 dni od resetu

Tabela 4.1: Publiczne metody klasy DatabaseHandler

4.2 Reset sezonowy

```
@tasks.loop(hours=24)
async def season_reset_check():
    if db.needs_reset():
        db.reset_season()           # zeruje punkty i gry
        logger.info("Sezon zresetowany.")
        for guild in bot.guilds:
            await stats.clear_roles(guild) # odbierz role
            Discord
```

Listing 4.1: Automatyczny reset co 10 dni

5. Voice API (TTS)

5.1 System ogłoszeń głosowych

Bot używa dwupoziomowego systemu TTS:

1. **gTTS** (Google Text-to-Speech) — wysoka jakość głosu polskiego, wymaga połączenia z internetem.
2. **pyttsx3** (fallback) — lokalny syntezytor Windows SAPI5, działa offline gdy Google jest niedostępne.

```
async def announce(self, text: str) -> None:
    try:
        # Proba gTTS (timeout 6s)
        await asyncio.wait_for(
            loop.run_in_executor(None, _gtts_save, text, path),
            timeout=6,
        )
    except Exception:
        # Fallback do lokalnego pyttsx3
        await loop.run_in_executor(None, _pyttsx3_save, text,
            path)
```

Listing 5.1: Logika wyboru silnika TTS

Wywołanie `gTTS.save()` jest blokujące (wywołanie sieciowe) i musi być uruchamiane w osobnym wątku przez `run_in_executor` — inaczej blokuje pętlę `asyncio` i powoduje opóźnienia w całej grze.

5.2 Ogłoszenia w grze

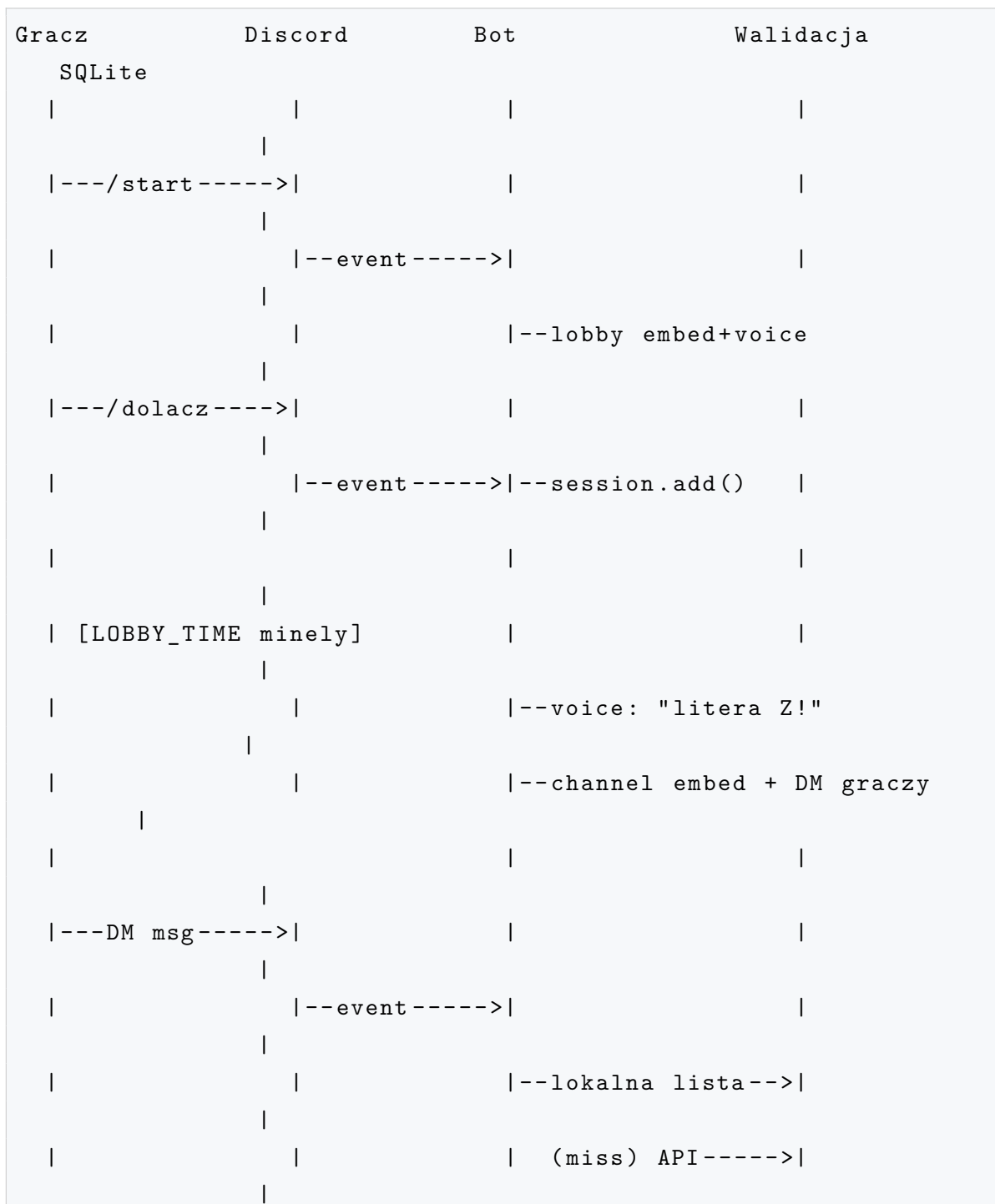
Zdarzenie	Ogłoszenie
Start lobby	„Gra rozpocznie się za <i>N</i> sekund. Dołącz wpisując komendę <i>dołącz!</i> ”
Start gry	„Gra się zaczyna!”
Ogłoszenie rundy	„Runda <i>X</i> z <i>Y</i> . Wylosowana litera to: <i>Z</i> . Macie <i>N</i> sekund!”
Ostrzeżenie 50%	„Zostało [<i>czas</i>]! Śpieszcie się!”
Ostrzeżenie 15%	„Zostało [<i>czas</i>]! Śpieszcie się!”
Zwycięzca rundy	„Zwycięzca rundy: [<i>imię</i>]!”
Koniec gry	„Koniec gry! Zwycięzcą zostaje: [<i>imię</i>], z wynikiem <i>N</i> punktów.”

Tabela 5.1: Ogłoszenia głosowe bota

5.3 Kolejność zdarzeń w rundzie

1. Bot **mówi głosowo** treść rundy (litera, czas) — czeka aż skończy.
2. **Jednocześnie** wysyłany jest embed na kanał tekstowy i DM do graczy.
3. Proporcjonalne przypomnienia głosowe + DM w trakcie odliczania.
4. Runda kończy się gdy **wszyscy** gracze wyślą odpowiedzi (przed upływem czasu).

6. Przepływ danych — diagram sekwencji



```
|          |          |<-True/False-----|
|          |          |
|          |          |--wyniki embed    |
|          |          |
|          |          |--faza zaskarzania
|          |          |
|          |          |--save_stats()
|          |          | |
|          |          |--update_roles() |
|          |          |
```

Listing 6.1: Uproszczony przepływ jednej rundy

A. Zależności projektu

```
discord.py>=2.3.0
discord.py[voice]
gTTS>=2.3.0
pyttsx3>=2.90
SQLAlchemy>=2.0.0
python-dotenv>=1.0.0
PyNaCl>=1.5.0
aiohttp>=3.9.0
```

Listing A.1: requirements.txt

Biblioteka	Zastosowanie
discord.py	Interfejs Discord: komendy, embedy, DM, voice, UI buttons, role
gTTS	Text-to-Speech po polsku (wymaga internetu, Google API)
pyttsx3	Lokalny TTS fallback (Windows SAPI5, offline)
SQLAlchemy	ORM dla bazy SQLite, modele Mapped[]
python-dotenv	Wczytywanie zmiennych z pliku .env
PyNaCl	Wymagane przez discord.py dla voice
aiohttp	Asynchroniczne HTTP (używane przez discord.py i walidację)

Tabela A.1: Zależności i ich zastosowanie